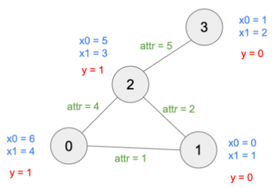
GNN

In pytorch……

- Graph
  - x → node feature
  - edge_index
  - edge_attr → edge feature
  - y → label, output value

```
Data(x=[4, 2], edge_index=[2, 8], edge_attr=[8, 1], y=[4, 1])
```



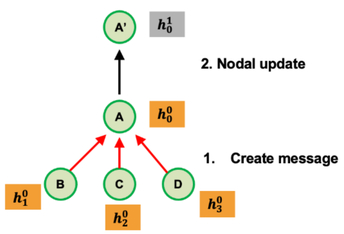

```
x = torch.tensor(
    [[6, 4],
     [0, 1],
     [5, 3],
     [1, 2]]
)
```

```
edge_attr = torch.tensor(
    [[1],
     [1],
     [4],
     [4],
     [2],
     [2],
     [5],
     [5]]
)
```

```
edge_index = torch.tensor(
    [[0, 1, 0, 2, 1, 2, 2, 3],
     [1, 0, 2, 0, 2, 1, 3, 2]]
)
```

```
y = torch.tensor(
    [[1],
     [0],
     [1],
     [0]]
)
```


Message Passing mechanism



1.

$$m_{0,1}^0 = 0.5h_0^0 + 2h_1^0$$

$$m_{0,2}^0 = 0.5h_0^0 + 2h_2^0$$

$$m_{0,3}^0 = 0.5h_0^0 + 2h_3^0$$

$$m_0^0 = aggr\,(m_{0,1}^0, m_{0,2}^0, m_{0,3}^0)$$

$$aggr = \{max, add, mean\}$$

2.

$$h_0^1 = h_0^0 + 0.5m_0^0$$

Message Passing in Pytorch

1.

$$m_{0,1}^0 = 0.5h_0^0 + 2h_1^0$$

$$m_{0,2}^0 = 0.5h_0^0 + 2h_2^0$$

$$m_{0,3}^0 = 0.5h_0^0 + 2h_3^0$$

$$m_0^0 = aggr\,(m_{0,1}^0, m_{0,2}^0, m_{0,3}^0)$$

$$aggr = \{max, add, mean\}$$

2.

$$h_0^1 = h_0^0 + 0.5m_0^0$$

```
# define message passing layer
self_designed_mp_layer = self_designed_MessagePassingLayer(aggr='min')
```

```
# Go through 1 message passing:
graph = Data(x=x, edge_index=edge_index, edge_attr=edge_attr, y=y)
```

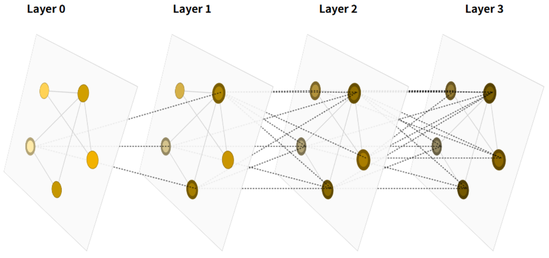
```
class self_designed_MessagePassingLayer(nn.MessagePassing):
  def __init__(self, aggr='max'):
        super(self_designed_MessagePassingLayer, self).__init__()
        self.aggr = aggr

    def forward(self, x, edge_index):
        return self.propagate(edge_index, x=x)

    def message(self, x_i, x_j):
        return 0.5 * x_i + 2 * x_j

    def update(self, aggr_out, x):
        return x + 0.5 * aggr_out
```

```
graph.x = self_designed_mp_layer(graph.x, graph.edge_index)
```

將data餵給model後 (forward)，propagate會自動呼叫 message & update

Graph Neural Networks

Neural networks have been adapted to leverage the structure and properties of graphs. We explore the components needed for building a graph neural network - and motivate the design choices behind them.



Hover over a node in the diagram below to see how it accumulates information from nodes around it through the layers of the network.

Graph Network-based simulators

https://arxiv.org/pdf/2002.09405

https://github.com/google-deepmind/deepmind-research/tree/master/learning_to_simulate

https://sites.google.com/view/learning-to-simulate/home#h.p_hjnaJ6k8y0wo

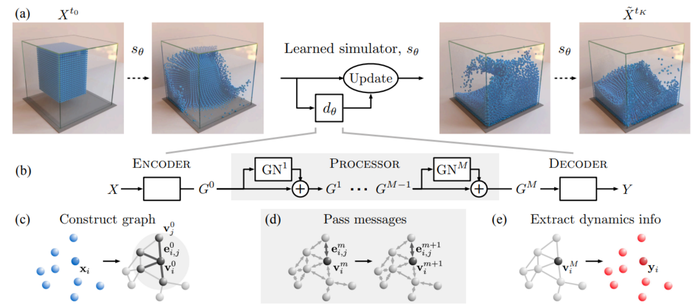
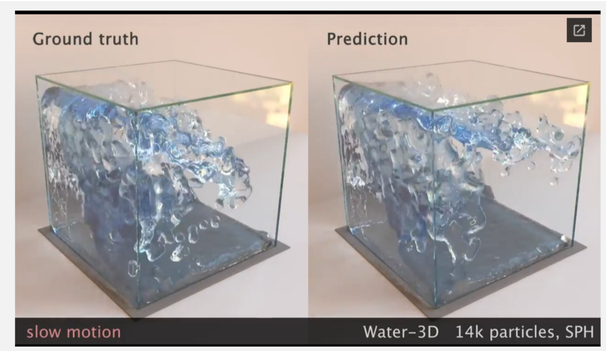


Figure 2. **(a)** Our GNS predicts future states represented as particles using its learned dynamics model, d_θ, and a fixed update procedure. **(b)** The d_θ uses an "encode-process-decode" scheme, which computes dynamics information, Y, from input state, X. **(c)** The ENCODER constructs latent graph, G^0, from the input state, X. **(d)** The PROCESSOR performs M rounds of learned message-passing over the latent graphs, $G^0, \ldots, G^M$. **(e)** The DECODER extracts dynamics information, Y, from the final latent graph, G^M.



MESHGRAPHNETS

https://github.com/deepmind/deepmind-research/tree/master/meshgraphnets.

https://sites.google.com/view/meshgraphnets

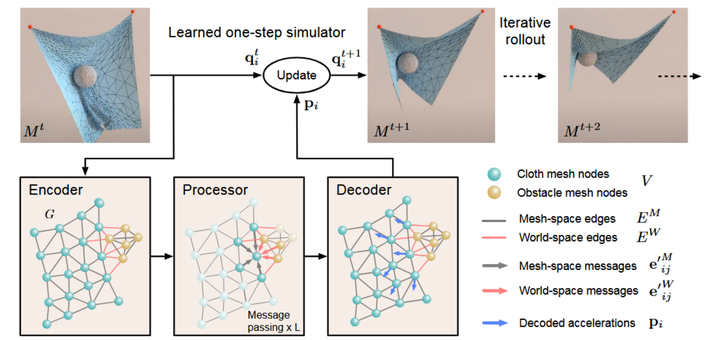


Figure 1: Diagram of MESHGRAPHNETS operating on our SPHEREDYNAMIC domain (video). The model uses an Encode-Process-Decode architecture trained with one-step supervision, and can be applied iteratively to generate long trajectories at inference time. The encoder transforms the input mesh M^t into a graph, adding extra world-space edges. The processor performs several rounds of message passing along mesh edges and world edges, updating all node and edge embeddings. The decoder extracts the acceleration for each node, which is used to update the mesh to produce M^{t+1}.

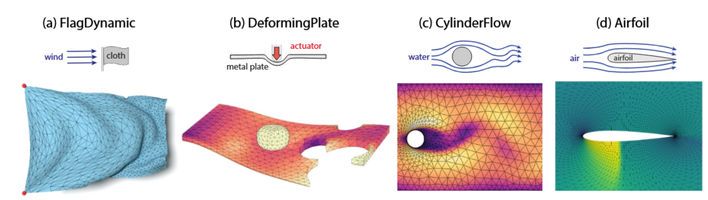


Figure 2: Our model can predict dynamics of vastly different physical systems, from structural mechanics over cloth to fluid dynamics. We demonstrate this by simulating (a) a flag waving in the wind, (b) a deforming plate, (c) flow of water around a cylinder obstacle, and (d) the dynamics of air around the cross-section of an aircraft wing (videos). The color map shows the von-Mises stress in (b), and the x-component of the velocity field in (c),(d).

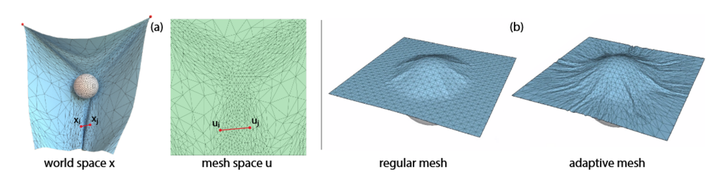
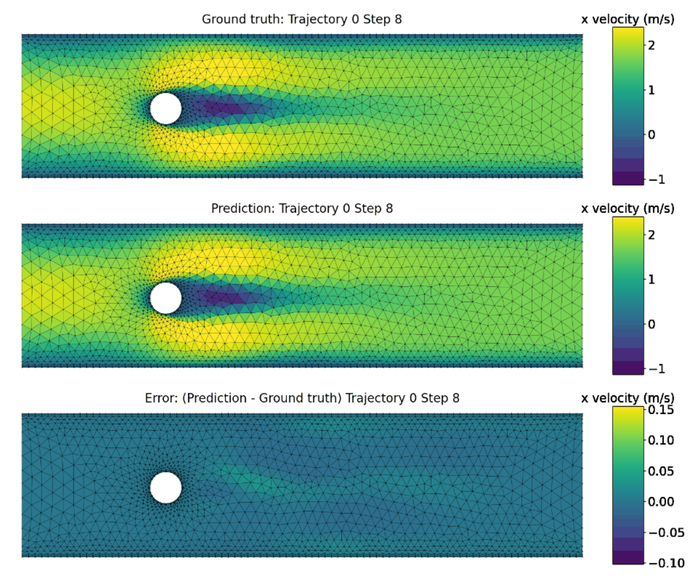



Figure 3: Simulation of a cloth interacting with a sphere. (a) In red, we highlight two nodes which are close in world-space but far in mesh-space, between which a world edge may be created. (b) With the same number of nodes, adaptive remeshing enables significantly more accurate simulations than a regular mesh with the same number of nodes.